COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 23, 2026	**DATE SUBMITTED:**	July 28, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **Hands-On Activity 3.1 Linked List**

1. PROCEDURE OUTPUT

Screenshot

```
#include <iostream>
#include "singly_ll.h"

class Node {
public:
    char data;
    Node* next;
};

int main() {
    // Creating the nodes
    Node* head = nullptr;
    Node* second = nullptr;
    Node* third = nullptr;
    Node* fourth = nullptr;
    Node* fifth = nullptr;
    Node* last = nullptr;

    // Create data allocation
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;

    // Define the 1st Node
    head->data = 'C';
    head->next = second;

    // Define the 2nd Node
    second->data = 'P';
    second->next = third;

    // Define the 3rd Node
    third->data = 'E';
    third->next = fourth;

    // Define the 4th Node
    fourth->data = '0';
    fourth->next = fifth;

    // Define the 5th Node
    fifth->data = '1';
    fifth->next = last;

    last->data = '0';
    last->next = nullptr;

}
```

```
PS C:\Users\LENOVO\Documents\CPE010_COMPILATION>
```

Discussion

In this implementation of the code, there are **no source code** which **connects to the functions inside the class** which **this code is just the implementation what's the nodes**

	inside and what's the data inside each nodes. Therefore, the result is just **blank** unless, **you implement the listTraversal function in the main driver code** which **displays the current Linked List which is CPE010**.

Table 3-1. Output of Initial/Simple Implementation

Operations	Screenshot

Traversal	`template <typename T>`
	`void listTraversal(SingleList<T>* head) {`
	`  while (head!= nullptr){`
	`  std::cout << head -> data;`
	`  // to check if the next line is not equal to null`
	`  if (head -> next != nullptr) {`
	`  std::cout << "->";`
	`  }`
	`  head = head -> next;`
	`  }`
	`  std::cout << std::endl;`
	`}`
	
	`std::cout << "Linked List Traversal: ";`
	`listTraversal(head);`

Insertion at Head	`template <typename T>`
	`void sllinsertHead(T newData, SingleList<T>** currentHead){`
	`  //Allocate the memory for the new node`
	`  SingleList<T>* newNode = new SingleList<T>;`
	`  //Put our data into the new node`
	`  newNode -> data = newData;`
	`  //Set Next of the new node to point to the previous head`
	`  newNode -> next = *currentHead;`
	`  //reset the head to point to the new node`
	`  *currentHead = newNode;`
	`}`
	
	`std::cout << "Newly-Inserted head of Linked List: ";`
	`sllinsertHead(std::string("G"), &head);`
	`listTraversal(head);`

Insertion at any part of the list

```
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T> * prevNode){
    //Check if it is the head node (previous node is null)
    if (prevNode == nullptr){
       std::cout<<"Previous Node cannot be null."<<std::endl;
       return;
    }

    //Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //Store data in the new node
    newNode -> data = newData;
    //Point new node to the node previous node was pointing to next
    newNode -> next = prevNode -> next;
    //Point previous node to the new node
    prevNode -> next = newNode;

}
```

```
std::cout << "After inserting E after P: \n";
sllGeneralinsertion(std::string("E"), second);
listTraversal(head);
```

Insertion at the end

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){

    //Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //Dereference to the head node
    SingleList<T>* currentNode = *head;
    //store data to the new node
    newNode -> data = newData;
    //point next of new node to NULL
    newNode -> next = nullptr;

    //Check if the list is empty
    if (currentNode == nullptr){
        currentNode == newNode;
        return;
    }

    // Traverse the list until next of the node is NULL
    while (currentNode -> next != nullptr) {
        //Iteration going to one node to another
        currentNode = currentNode -> next;

    }
    currentNode -> next = newNode;
}
```

```
std::cout << "Inserting a new node at the end of the Linked List: ";
sllinsertEnd(std::string("Z"), &head);
listTraversal(head);
```

Deletion of the node

```
template <typename T>
void sllDelete(T findData, SingleList<T>** head){
    //Check if there is a list
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //Search for the node:
    while(currNode != nullptr && currNode -> data != findData){
       prevNode = currNode;
       currNode = currNode -> next;
    }

   //if data is not found
   if(currNode == nullptr) return;

   //delete the head node (the head node is the findData)
   if (prevNode == nullptr){
       *head = currNode -> next;
   }else{
       prevNode -> next = currNode -> next;
   }

   delete currNode;
}
```

```
//Deleting the node with data C
std::cout << "deleting a node: \n";
std::cout << "Before deleting C: \n";
listTraversal(head);

sllDelete(std::string("C"), &head);

std::cout << "After deleting C: \n";
listTraversal(head);

//Deleting the node with data P
std::cout << "deleting a node: \n";
std::cout << "Before deleting P: \n";
listTraversal(head);

sllDelete(std::string("P"), &head);

std::cout << "After deleting P: " <<std::endl;
listTraversal(head);
```

Table 3-2. Code for the List Operations

a.

Source Code	`std::cout << "Linked List Traversal: ";` `listTraversal(head);`
Console	`Linked List Traversal: C->P->E->0->1->0`

b.

Source Code	`std::cout << "Newly-Inserted head of Linked List: ";`
	`sllinsertHead(std::string("G"), &head);`
	`listTraversal(head);`

Console	`Newly-Inserted head of Linked List: G->C->P->E->0->1->0`

c.

Source Code	`std::cout << "After inserting E after P: \n";`
	`sllGeneralinsertion(std::string("E"), second);`
	`listTraversal(head);`

Console	After inserting E after P: G->C->P->E->E->0->1->0

d.

Source Code	`std::cout << "Inserting a new node at the end of the Linked List: ";`
	`sllinsertEnd(std::string("Z"), &head);`
	`listTraversal(head);`

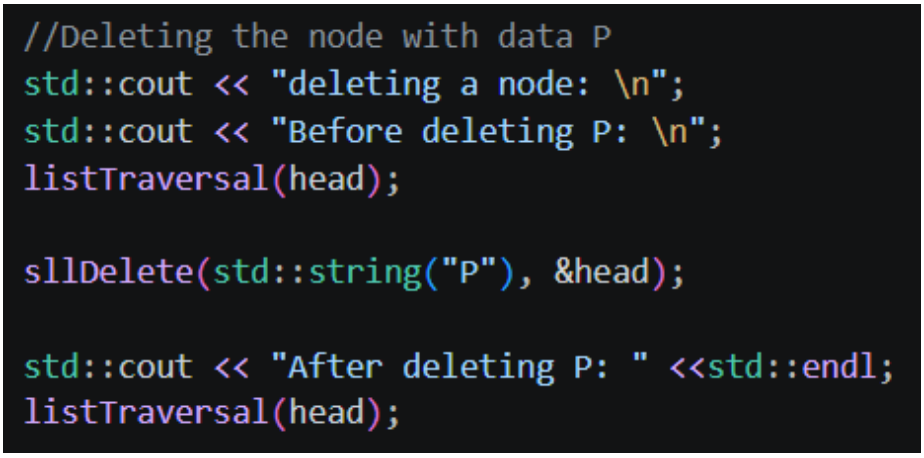
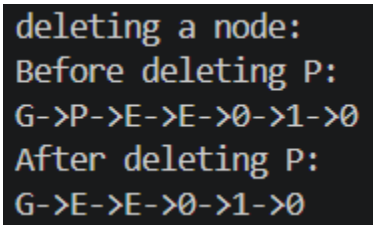
Console	Inserting a new node at the end of the Linked List: G->C->P->E->0->1->0->Z

e.

Source Code	`//Deleting the node with data C`
	`std::cout << "deleting a node: \n";`
	`std::cout << "Before deleting C: \n";`
	`listTraversal(head);`
	
	`sllDelete(std::string("C"), &head);`
	
	`std::cout << "After deleting C: \n";`
	`listTraversal(head);`

Console	deleting a node: Before deleting C: G->C->P->E->E->0->1->0 After deleting C: G->P->E->E->0->1->0

f.

Source Code	
Console	

```
//Deleting the node with data P
std::cout << "deleting a node: \n";
std::cout << "Before deleting P: \n";
listTraversal(head);

sllDelete(std::string("P"), &head);

std::cout << "After deleting P: " <<std::endl;
listTraversal(head);
```

```
deleting a node:
Before deleting P:
G->P->E->E->0->1->0
After deleting P:
G->E->E->0->1->0
```

Table 3-3. Code and Analysis for Singly Linked Lists

SCREENSHOT(s)	ANALYSIS
	In this **dllTraverse function**, it starts to the **Forward Traversal** which the **loop starts with the currentNode (head) and prints all the node's data until the nullptr** by the use of the codes **"while (currentNode != nullptr) " and "std::cout << currentNode -> data << "->"; tail - currentNode; currentNode = currentNode -> next"** which **it saves also the last node in tail**. In **Reverse Transversal**, the **loop starts at the tail and prints all the node's data too until the nullptr** by the use of the codes **"while (tail != nullptr)" and "std::cout << tail-> data << "->"; tail = tail -> prev;"**. In the **Main Driver Code**, it is **just the "Initial Traversal: \n"** and **the main function caller "dllTraverse(head)"**. Lastly, the **output shown the forward and reverse traversal of the node's data**.

```
template<typename T>
void dllTraverse(doubleList<T>* currentNode) {
    doubleList<T>* tail;

    std::cout << "Forward \n";
    //start to end
    while (currentNode != nullptr) {
        std::cout << currentNode->data << "->";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    std::cout<<std::endl;

    std::cout << "Reverse \n";
    //end to start
    while (tail != nullptr) {
        std::cout << tail->data << "->";
        tail = tail->prev;
    }
};
```

```
std::cout << "Initial Traversal: \n";
dllTraverse(head);
```

```
Initial Traversal:
Forward
C->P->E->
Reverse
E->P->C->
```

```
template<typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead) {
    doubleList<T>* newNode = new_Node(newData);
    newNode->next = *currentHead;
    if (*currentHead != nullptr) {
        (*currentHead)->prev = newNode;
    }
    *currentHead = newNode;
}
```

```
std::cout << "\n";
std::cout << "Inserting at the head: " << std::endl;
dllInsertHead('G', &head);
dllTraverse(head);
```

```
Inserting at the head:
Forward
G->C->P->E->
Reverse
E->P->C->G->
```

In the **dllInsertHead Function**, we need to create a **new node to allocate new data to the memory** by this implementation **"doubleList<T>* newNode = new_Node(newData)"**. Also, we need to **link the new node to the current head because we need to insert the new head** with this code **(newNode->next = currentHead)** which **points the new node's next pointer to the old head**. Moreover, we need now to **update the old head's previous pointer which we need to see if the list isn't empty** by this code **"if (*currentHead != nullptr)" and "(*currentHead) -> prev = newNode;"**. Now, we **move the head pointer to the new node that we created** by this code **"*currentHead = newNode;"** which **the new node becomes the new head**. In the **main driver code**, we need to **call the dllInsertHead function before the dllTraverse function because the dllInsertHead function contains with the parameters 'G' which is the node's data and &head which is the current address of the pointer** so the **function can reassign the head itself**. Lastly, the **output shows the output that the G added on the list which makes it a new head**.

```
template<typename T>
void dllInsertTail(T newData, doubleList<T>** currentHead) {
    doubleList<T>* newNode = new_Node(newData);

    if (*currentHead == nullptr) {
        *currentHead = newNode;
        return;
    }

    doubleList<T>* currentNode = *currentHead;
    while (currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }
    currentNode->next = newNode;
    newNode->prev = currentNode;
}
```

```
std::cout << "\n";
std::cout << "Inserting at the end: " << std::endl;
dllInsertTail('1', &head);
dllTraverse(head);
```

In the **dllInsertTail function**, we need to **create a new node again**, then **we need to check if the list is empty before we traverse to the last node** by this loop **"doubleList<T>* currentNode = *currentHead; while (currentNode->next != nullptr) { currentNode = currentNode->next; }"** which it **loops through the list until it reaches to the tail**. Next, we need to **link the currentNode to the newNode** by this code **"currentNode->next = newNode; newNode->prev = currentNode;"** which **connects the new node at the end and it will update the previous pointer**. In the **main driver code**, it is the **same as the dllInsertHead which outputs the 1 at the end but there's a NULL which indicates that the list terminates properly**.

```
Inserting at the end:
Forward
G->C->P->E->1->NULL

Reverse
1->E->P->C->G->NULL
```

```
// GENERAL INSERTION
template<typename T>
void dllInsertAny(doubleList<T>* prevNode, T newData) {
    //Check if it is the head node (previous node is null)
    if (prevNode == nullptr) {
        std::cout << "The previous node cannot be NULL.\n";
        return;
    }

    //Allocate a new node
    doubleList<T>* newNode = new_Node(newData);

    // Link new node with next node
    newNode->next = prevNode->next;

    // Link new node with previous node
    newNode->prev = prevNode;

    // Link previous node with new node
    prevNode->next = newNode;

    // Fix the backward link of the next node
    if (newNode->next != nullptr) {
        newNode->next->prev = newNode;
    }
}
```

```
std::cout << "\n";
std::cout << "General Insertion: " << std::endl;
doubleList<char>* current = head;

while (current != nullptr && current->data != 'P'){
    current = current->next;
}

if(current != nullptr){
    dllInsertAny(current, 'X');
}

dllTraverse(head);
```

```
General Insertion:
Forward
G->C->P->X->E->1->NULL

Reverse
1->E->X->P->C->G->NULL
```

In the **General Insertion** which is **dllInsertAny function**, we need first to **validate if the previous node is null** by this code **"if (prevNode == nullptr) { std::cout<< "The previous node cannot be NULL. \n"; return; }"** because **it prevents insertion if the reference node have no data inside it**. Next, we **create a node like the usual implementation which we connect the new node between the prevNode and its next node** by this **code "newNode->next = prevNode->next; prevNode->next = newNode;"**. Now, **we linked it backwards** by this code **"newNode ->prev = prevNode; if (newNode->next != nullptr) newNode->next->prev = newNode;"** which it **enables the reverse transversal**. In the **main driver code**, the **driver searches for the node containing 'P'** by this code **"while (current != nullptr && current->data != 'P') { current = current->next; }"**, then **we can insert the new data** by **dllInsertAny** which has the **parameters current (current->next pointer) and 'X' is the new data**. The **output is we correctly inserted 'X' after 'P' and we managed to enable both forward and reverse traversals**.

```
//DELETION
template<typename T>
void dllDeleteNode(doubleList<T>** head, doubleList<T>* delNode) {
    // If list is empty or node to delete is NULL, do nothing
    if (*head == nullptr || delNode == nullptr)
        return;
    // If the node to delete is the head, move head to the next node
    if (*head == delNode)
        *head = delNode->next;
    // If there is a next node, update its prev pointer
    if (delNode->next != nullptr)
        delNode->next->prev = delNode->prev;
    // If there is a previous node, update its next pointer
    if (delNode->prev != nullptr)
        delNode->prev->next = delNode->next;
    // Delete the node and clear pointer
    delete delNode;
    delNode = nullptr;
}
```

```
    std::cout << "\n";
    std::cout << "Deleting a node: " << std::endl;
    dllDeleteNode(&head, second);
    dllTraverse(head);
```

```
Deleting a node:
Forward
G->C->X->E->1->NULL

Reverse
1->E->X->C->G->NULL
```

In the **dllDeleteNode function**, first we must **validate if the list is empty or the reference node is NULL** by this code **"if (*head == nullptr || delnode == nullptr) return;"** which the **delnode is a reference pointer that you want to delete**. Next, **we need to move the head to the next node** by this code **"if (*head == delnode) *head = delnode -> next;"**. Now, **we need to know if there us a next node to update its prev pointer** by this code **"if (delNode -> next != nullptr) delnode -> next ->prev = delnode -> prev;"** which **ensures also the next node's pointer skips over the deleted node**. Also, **we need to know if there is a previous node to update its next pointer** by this code **"if (delNode -> prev != nullptr) delNode -> prev -> next = delNode -> next;"** which **ensures the previous node's next pointer skips the deleted node**. Now, **we need to delete the node and clear the pointer** by this code **"delete delNode; delNode = nullptr"**. In the main driver code, **dllDeleteNode with the parameters &head which is the address of the pointer you want to delete** and **Second which is in the list it is the 'P' that you want to delete**. Lastly, the **output shows that it successfully deletes the 'P' in the list and enables the forward and reverse traversals successfully**.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.
- The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

CODE:

Main Driver

```cpp
#include <iostream>
#include "doubly_ll.h"

int main() {
    song<std::string>* head = new_Node(std::string("Kung Wala ka by Hale"));
    song<std::string>* second = new_Node(std::string("Dating Tayo by Tj Monterde"));
    song<std::string>* third = new_Node(std::string("About You by The 1975"));

    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    third->next = head; // Circular Linked List
    head->prev = third; // Circular Linked List

// Display the playlist

    std::cout << "\n";
    std::cout << "\n---Sadboi Playlist---\n";
    dllplayAll(head);

// Inserting a new song after the second song
    std::cout << "\n";
    std::cout << "Inserting at the end: " << std::endl;
    dllInsertAfter(third, std::string("Mundo by IV of Spades"));
    dllplayAll(head);

//Deleting a node

    std::cout << "\n";
    std::cout << "\nDeleting a node:\n";
    song<std::string>* toDelete = nullptr;
    song<std::string>* current = head;
    do {
        if (current->data == "Dating Tayo by Tj Monterde") {
            toDelete = current;
            break;
        }
        current = current->next;
    } while (current != head);
```

```
42      } while (current != head);
43
44      if (toDelete != nullptr) {
45          dllDeleteNode(&head, toDelete);
46      }
47
48      dllplayAll(head);
49
50  //Play Next Song
51      std::cout << "\n";
52      std::cout << "Play Next Song: " << std::endl;
53      song<std::string>* currentSong = head;
54      dllplayNext(&currentSong);
55      dllplayAll(currentSong);
56
57  //Play Previous Song
58      std::cout << "\n";
59      std::cout << "Play Previous Song: " << std::endl;
60      song<std::string>* currentSongPrev = currentSong->prev;
61      dllplayPrevious(&currentSongPrev);
62      dllplayAll(currentSongPrev);
63
64      return 0;
65  }
```

Doubly_ll.h Code:

```
#ifndef DOUBLE_LL_PLAYL_H
#define DOUBLE_LL_PLAYL_H

template<typename T>
class song {
public:
    T data;
    song<T>* next = nullptr;
    song<T>* prev = nullptr;
};

template<typename T>
song<T>* new_Node (T newData) {

    song<T>* newSong = new song<T>;

    newSong->data = newData;
    newSong->next = newSong; //Circular Linked List
    newSong->prev = newSong;

    return newSong;
}

template<typename T>
void dllplayAll(song<T>* currentNode) {

    if (currentNode == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }
    song<T>* current = currentNode;
    do {
        std::cout << current->data << "->";
        current = current->next;
    }while (current != currentNode);
    std::cout << "END\n";
}


// GENERAL INSERTION
template<typename T>
void dllInsertAfter(song<T>* prevNode, T newData) {
```

```cpp
void dllInsertAfter(song<T>* prevNode, T newData) {
    //Check if it is the head node (previous node is null)
    if (prevNode == nullptr) return;
    song<T>* newSong = new_Node(newData);
    newSong->next = prevNode->next;
    newSong->prev = prevNode;
    prevNode->next->prev = newSong;
    prevNode->next = newSong;
   }


//DELETION
template<typename T>
void dllDeleteNode(song<T>** head, song<T>* delNode) {
    // If list is empty or node to delete is NULL, do nothing
    if (*head == nullptr || delNode == nullptr)
        return;
    // Case: only one node in the list
   if (delNode->next == delNode && delNode->prev == delNode) {
       *head = nullptr;
   } else {
       // Update the next and prev pointers of neighboring nodes
       delNode->prev->next = delNode->next;
       delNode->next->prev = delNode->prev;

       // If the node to be deleted is the head, update the head pointer
       if (*head == delNode) {
           *head = delNode->next;
       }
   }
    delete delNode;
    delNode = nullptr;
}

//Play Next Song
template<typename T>
void dllplayNext(song<T>** currentNode) {
   if (*currentNode == nullptr) {
       std::cout << "No song is currently playing.\n";
       return;
    }
    *currentNode = (*currentNode)->next;
    std::cout << "Now playing from the Playlist: " << (*currentNode)->data << std::endl;
}

//Play Previous Song
template<typename T>
void dllplayPrevious(song<T>** currentNode) {
    if (*currentNode == nullptr) {
        std::cout << "No song is currently playing.\n";
        return;
    }
    *currentNode = (*currentNode)->prev;
    std::cout << "Now playing from the Playlist: " << (*currentNode)->data << std::endl;
}

#endif
```

OUTPUT:

```
---Sadboi Playlist---
Kung Wala ka by Hale->Dating Tayo by Tj Monterde->About You by The 1975->END

Inserting at the end:
Kung Wala ka by Hale->Dating Tayo by Tj Monterde->About You by The 1975->Mundo by IV of Spades->END


Deleting a node:
Kung Wala ka by Hale->About You by The 1975->Mundo by IV of Spades->END

Play Next Song:
Now playing from the Playlist: About You by The 1975
About You by The 1975->Mundo by IV of Spades->Kung Wala ka by Hale->END

Play Previous Song:
Now playing from the Playlist: Mundo by IV of Spades
Mundo by IV of Spades->Kung Wala ka by Hale->About You by The 1975->END
```

Conclusion and Lessons Learned	**In Conclusion**, I've learned how to create a **doubly linked list** now which the **difference between the singly linked list is that there is previous and next which has now forward and reverse traversal on it** because **it can go on any direction now**. **Doubly Linked List has different implementation of the functions because now it needs to reference the previous node, current node, and the next node to execute the given functions**. There are some implementations which are the same as singly like **allocating a new node for the list and creating nodes inside of the list are the same**. Also, I've learned in the supplementary on how to do a **Circular Doubly Linked List** which is **another type of Doubly Linked List** where now is the direction is like a **CLOSED LOOP** because the **last node connects back to the first and the first node back to the last node**. This Linked List has **different kind of approach in implementing the data on each nodes** because **it needs to close its loop to connect the last node to the first and first node to the last**. Also, it has **different approach in functions like the dllDeleteNode function** where the code has **Do While loop in the main driver code** to **ensure that the condition executes at least once, so the head node itself is checked first before moving forward because the list is Circular so the loops stops when it cycles back to head which is the target song** and it was initialized by this code **"do {if (current->data == "Dating Tayo by Tj Monterde") { toDelete = current; break; } current = current->next; } while (current != head);"**. Also, in the **main function code**, **dllDeleteNode code have been modified because it has the same safety check in doubly "if (*head == nullptr || delNode == nullptr) return;" which ensures if the list is empty or the reference node is NULL**. Next, is **we have a case condition if the node points itself both directions, it's the one and only node so that we can delete it completely on the lis**t and it was initialized by this code **"if (delNode->next == delNode && delNode->prev == delNode) { *head = nullptr; }"**. Now, we have **the general case which the previous node's next skips over the delNode and next node's prev skips over delNode** by this code **"delNode->prev->next = delNode->next; delNode->next->prev = delNode->prev;"**. Next, we **ensures that the head pointer moves forward if the deleted node was the head itself** by this code **"if (*head == delNode) { *head = delNode->next; }", then we delete the code by "delete delNode; delNode = nullptr"**. There is lot of changes and modifications in Circular Linked Lists **because of its logic** where now **it is**

	a closed loop where you must connect the last node to the first and first node to the last which makes it organized and easily to manipulate or understand.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Microsoft Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts in Doubly Linked List and Circular Doubly Linked List**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It helps me understand the concepts of the doubly linked list and circular doubly linked list implementation of the different operations well.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.